

GEII Annecy- Années 2025/2026

Parcours Electronique et Systèmes Embarqués (ESE)

PORTFOLIO

Réalisation d'un ordinateur de bord pour voiture de course

Auteur : GAVET Romain

Projet SAé

Contact : Romain.Gavet@etu.univ-smb.fr

Lien vers le Drive avec tous les fichiers :

<https://drive.google.com/drive/folders/1DhPBvFhPZ7Ev5jnKutqlR2fyGxq3myTQ?usp=sharing>



Table des matières

1.0-Introduction et contexte du semestre 4.....	3
2.0-Tableau de correspondance traces/critères.....	4
3.0-Rappel du cahier des charges (Semestre 4)	5
3.1-Prenant en compte la demande du client.....	5
3.2-Nouvelles fonctionnalités S4.....	5
3.3-Contraintes liées à la validation	6
3.4-Conception et modélisation du système.....	7
3.4.1-Respect des contraintes normatives	7
3.4.2-Schéma de conditionnement	7
3.4.3-Cas d'usage.....	7
4.0-Modèle dynamique.....	8
4.1-Modèle dynamique sans interruptions.....	8
5.0-Mise en place de la communication I2C	10
5.1-Justification des choix de conception.....	10
5.2-Utilisation des documentations techniques.....	10
5.2.1-Adresses I2C issues des datasheets	11
5.2.2-Sémantique des registres	11
5.3-Norme de configuration.....	11
6.0-Modèle fonctionnel.....	12
6.1-Méthode de découpage fonctionnelle.....	12
6.1.1-Gestion du capteur LM75	13
6.1.2-Gestion de l'écran LCD ST7032.....	13
6.2-Respectant les conventions de codage.	14
6.2.1 Convention de nommage.	14
7.0-Filtre numérique.....	15
7.1-Conception du filtre numérique passe bas du premier ordre	15
7.2-Characterisation du filtre obtenue.....	16
7.3-Débogage STM32.	16
7-4. Maitrise des temps d'exécution.....	18
7.4.1-Optimisation par le choix d'un filtre IIR du 1er ordre	18
7.4.2-Point d'amélioration identifié : mesure instrumentée du temps.....	19
8-0. Observations des tests.	19
8-1. Mise en place de test unitaire.....	19
8-1. Observations des trames en vérifiant le respect du protocole.	20
9-Plan de validation du système.	20

9.1-Schéma de câblage et justification	20
9.2-Matériel et environnement de test.....	21
9.2.1-Justification des étapes.	21
9.3-Justification des plages de validation.	22
9.4-Respect des règles normatives	22
9.5-Analyse des résultats	22
10-Analyse réflexive.	23

1.0-Introduction et contexte du semestre 4

Ce portfolio est la continuation du travail réalisé au semestre 3, dans le cadre du projet de SAé qui est la réalisation d'un ordinateur de bord pour voiture de course. Il documente les nouvelles compétences acquises et les nouvelles fonctionnalités lors du 4eme semestres.

Nous avons 4 compétences à démontrer étant : concevoir, implanter, vérifier et maintenir. Nous avons à notre disposition 4 documents avec les preuves de la conception, implantation des systèmes qui collent avec chacune des compétences.

Concevoir pour le dossier de conception, implanter pour le dossier de fabrication, maintenir avec la procédure de test et vérifier avec le rapport de test.

Les trois apprentissages critiques visés au semestres4 sont

- Mettre en œuvre une liaison série synchrone (protocole I2C)
- Mettre en œuvre un filtre numérique
- Mettre en œuvre une communication radio (RF)

Pendant ce semestre, nous allons ajouter deux systèmes principaux sur notre projet :

- La communication I2C avec deux composants : le capteur température LM75 et l'écran LCD ST0732, nous allons nous servir du capteur de température pour l'afficher en temps réel sur notre écran. Ainsi que l'accélérations que nous affichons sur notre écran LCD ST7032.
- Le filtre numérique passe bas du premier ordre, que nous avons réalisé avec la STM32 et le PicoScope dans le but de filtrer les signaux de l'ADLX-335, avec notre fréquence de coupure provenant du premier semestres $f_c = 600Hz$, avec $f_e = 10KHz$

2.0-Tableau de correspondance traces/critères

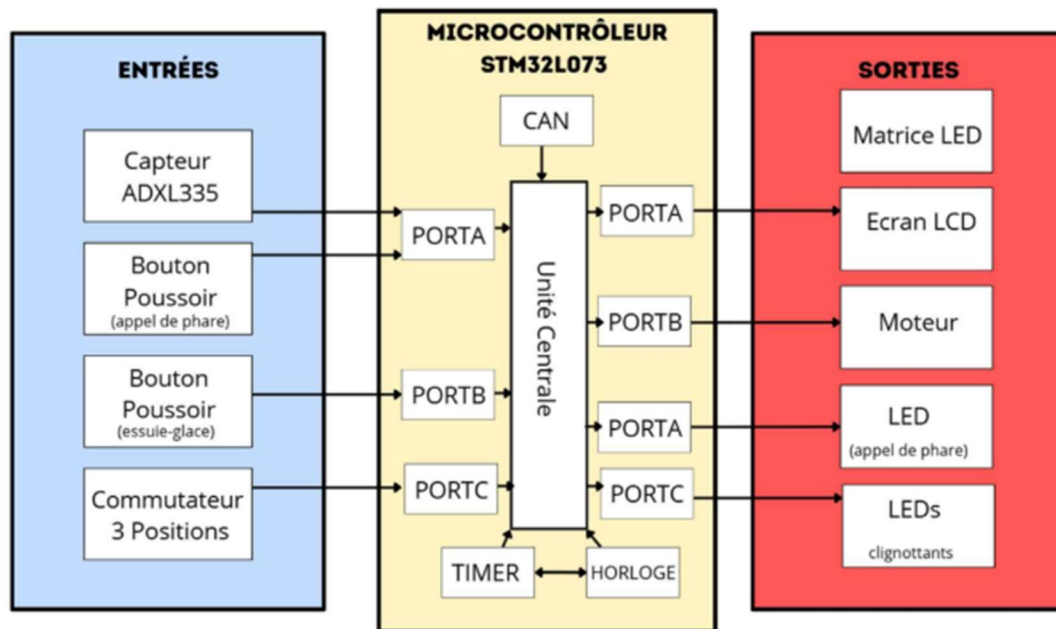
Le tableau ci-dessous correspond à la grille avec chaque critère d'exigences pour le semestre 4.

Savoir-faire:	Critères d'exigences	Quels traces?	Pages
Ecrire un cahier des charges	en prenant en compte la demande du client	Rappel du cahiers des charges	5
	en prenant en compte les contraintes liées à la validation	Rappel du cahiers des charges	6
	en respectant les contraintes normatives	Conception et modélisation du système	07-janv
Savoir-faire:	Critères d'exigences	Quels traces?	Pages
Concevoir un programme informatique	utilisant un modèle dynamique	Modèle dynamique	P8
	en justifiant les choix de conception	Justification choix de conception	10
	utilisation de doc technique de manière pertinente	Communication I2C	11-janv
	Respectant les normes de configuration	Communication I2C	P11
	méthode de découpage fonctionnelle	Modèle fonctionnel	12-13-14
	conventions de codage	Modèle fonctionnel	14
	démarche de conception du filtre	Filtre numérique	16
	en caractérisant le filtre	Filtre numérique	16-17
	Mettre en œuvre une manière de débogage	Filtre numérique	17-18
	Maîtrisant les temps d'exécution	Filtre numérique	19
	en vérifiant le respect du protocole	Communication I2C	20
	Mise en place de test unitaire	Tous les systèmes	20
Savoir-faire:	Critères d'exigences	Quels traces?	Pages
Procédure de Test	En indiquant et justifiant un schéma de câblages	Plan de validation du système	21
	En listant le matériel à tester et de test (logiciel)	Plan de validation du système	P21
	En détaillant et justifiant les étapes	Plan de validation du système	P21-22
	En indiquant et justifiant les plages de validité	Plan de validation du système	P22-23
Rapport de Test	En respectant les règles normatives	Plan de validation du système	P22-23
	En analysant les résultats	Plan de validation du système	23

3.0-Rappel du cahier des charges (Semestre 4)

Le cahier des charges du semestres 4 est la suite du semestre 3, nous allons ajouter comme énoncé plutôt la communication I2C, le filtrage.

Pour illustrer cela, nous avons réalisé le synoptique suivant ;



3.1-Prenant en compte la demande du client

Dans ce projet de SAÉ en BUT GEII, spécialité Électronique et Systèmes Embarqués (ESE), nous avons nous-mêmes endossé le rôle de client. À ce titre, nous avons défini le système que nous souhaitons concevoir, ce qui nous a amenés à élaborer un cahier des charges détaillé et adapté à notre projet.

Ce document précise les différentes fonctionnalités attendues ainsi que les sous-systèmes nécessaires à leur mise en œuvre. Nous y avons notamment défini les contraintes techniques, les interactions entre les composants, ainsi que les performances attendues, afin de structurer efficacement la conception et le développement du système.

Preuve dossier de conception

3.2-Nouvelles fonctionnalités S4

Communication I2C

L'objectif est d'établir une communication entre le microcontrôleur STM32 et les composants I2C :

- Le capteur de température LM75 : mesure la température et la transmet au STM32 sous forme d'un entier 16 bits.
- L'écran LCD ST7032 : reçoit les commandes d'initialisation et les données de texte à afficher. Il affiche en temps réel la température en degré (°C) et les vitesses calculées sur les axes X et Y de l'accéléromètre grâce à l'ADXL335.

Filtre numérique du premier ordre

Le filtre passe-bas analogique ($f_c = 600\text{Hz}$, $R = 10\text{k}\Omega$, $C = 20\text{nF}$) déjà implanté au semestres 3, sera complétement par celui numérique réalisé sur STM32. Ce filtre est appliqué aux signaux X et Y de l'accéléromètres avant l'acquisition des données.

Communication RF

Le cahier des charges prévoit également une transmission de données par radiofréquences, permettant d'envoyer ou de recevoir des informations à distance. Cette fonctionnalité a été intégré dans le synoptique du système (**voir Dossier de Fabrication p.4**).

Dans notre cas, nous aurons souhaité mettre en place la communication RF au sein d'un circuit automobile.

Pendant les courses automobiles les commissaires communiquent à la direction de course via une communication RF, et les informe en temps réels sur les incidents qu'il se déroule sur la piste.

Sur la piste les informations se transmettent via des drapeaux avec chacun leurs signification (drapeau bleu, jaune, rouge, orange et rouge...)

Notre système consistera à depuis se poste de contrôle, activer la matrice LEDs en bord de pistes qui annonce aux pilotes les informations grâce à la couleur de ces affichages.

3.3-Contraintes liées à la validation

Notre système comporte plusieurs contraintes pour sa validation.

- Le système doit pouvoir être testé de manière indépendante pour chaque sous-système (I2C, filtre).
- Les tests doivent être réalisables avec le matériel disponible à l'IUT (PicoScope 6, STM32).
- Toutes les valeurs attendues doivent être justifiées théoriquement et vérifiables expérimentalement.

Chaque système que nous avons réalisé comporte une validation différente des autres. Ces critères de validation ont été définis de manière à assurer la validation du système en intégralité.

Par exemple, le critère de validation des appels de phares étaient :

- Lors d'une appuie sur le bouton poussoir les clignotements doivent commencer.
- Clignotement de la LED « Appels de phares » pendant 3 secondes à une fréquence de 4Hz.

Ces critères de validation précis pour chaque système nous ont permis d'assurer le fonctionnement du projet.

Preuve Rapport de Test/procédure de Test

3.4-Conception et modélisation du système.

3.4.1-Respect des contraintes normatives

L'architecture matérielle du système s'articule autour du microcontrôleur STM32L073, qui joue le rôle de chef d'orchestre entre tous les périphériques. Sa synchronisation repose sur un timer cadencé par une horloge interne, garantissant la régularité des tâches temps réel : acquisition des données, calcul du filtre, gestion du PWM et des clignotements.

Du côté des entrées, deux voies distinctes alimentent le microcontrôleur en informations. Les signaux numériques des boutons poussoirs transitent par les GPIO, tandis que les grandeurs analogiques issues de l'accéléromètre ADXL335 et du capteur de température sont numérisées par le convertisseur ADC intégré. En sortie, trois coupleurs assurent respectivement la commande des LEDs, la communication avec l'écran LCD, et le pilotage du moteur d'essuie-glaces. Cette organisation reflète une séparation nette entre acquisition, traitement et actionnement.

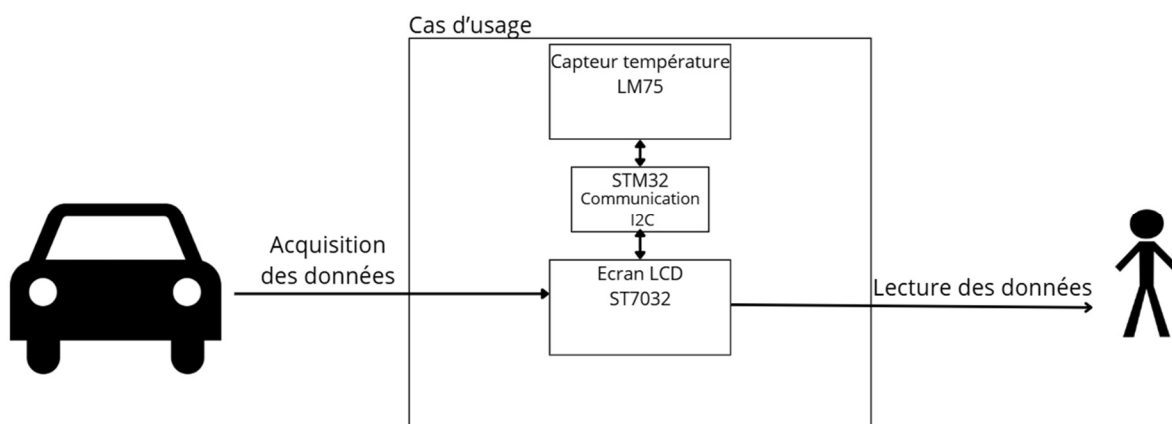
Preuve Dossier de Conception p11

3.4.2-Schéma de conditionnement

Le système repose sur un STM32L073RZTx alimenté en 3,3 V. La température ambiante est acquise via le capteur LM75 sur le bus I2C, et la vitesse simulée est déduite du traitement des données brutes de l'accéléromètre analogique ADXL335. L'écran LCD affiche ces informations en temps réel. Trois interfaces utilisateur complètent le tableau de bord : un commutateur trois positions pour les clignotants gauche/droit, un bouton pour les appels de phares, et un second bouton pour activer le moteur simulant les essuie-glaces.

3.4.3-Cas d'usage.

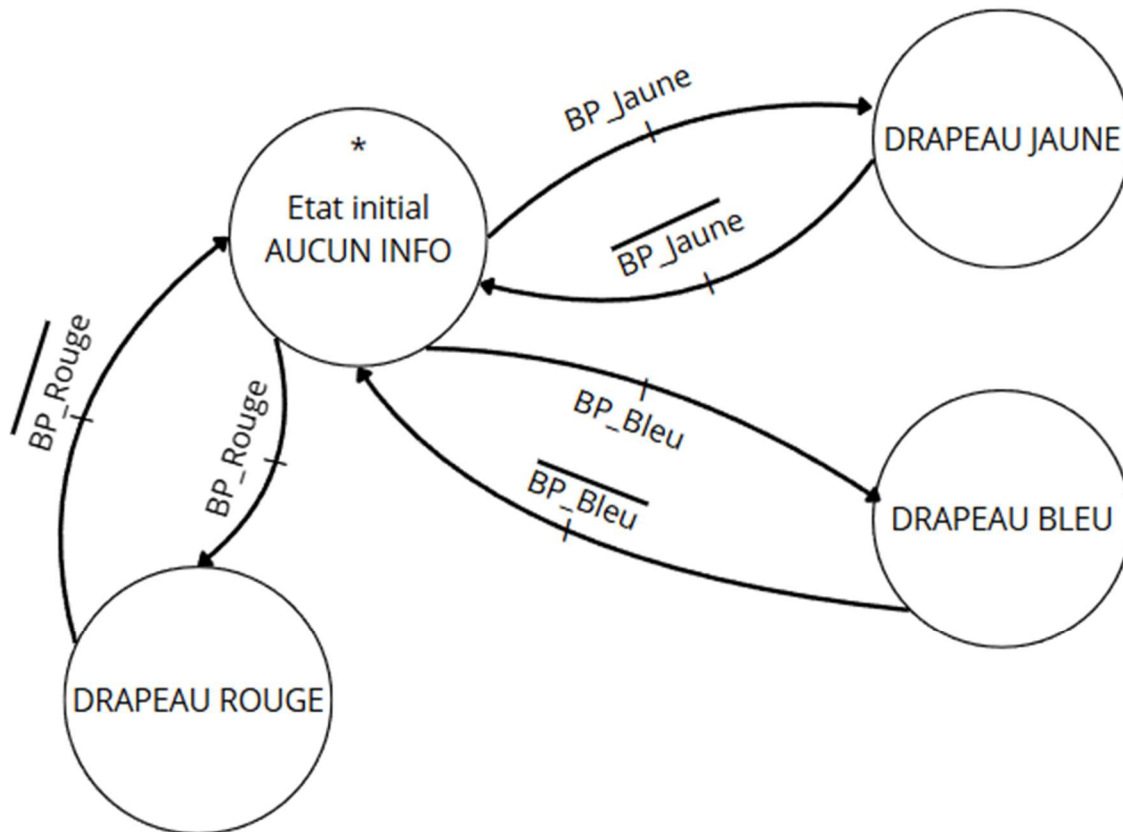
Pour représenter ces nouvelles fonctionnalités sur la STM32, nous pouvons réaliser ce cas d'usage suivant ;



4.0-Modèle dynamique.

4.1-Modèle dynamique sans interruptions.

Nous avons réalisé le modèle dynamique pour la communication RF (ci-dessous) que nous n'avons malheureusement pas eu le temps de réaliser.



Le modèle décrit le comportement de la communication RF, ici selon nos états, nos communications seront différentes. Nous avons plusieurs cas possibles ;

Dans le cas du drapeau jaune, nous souhaitons à distance allumer la matrice LEDs en jaune.

Idem pour le drapeau bleu et rouge, nous allumons la matrice LEDs en bleu ou en rouge.

Nous pouvons noter aussi le modèle dynamique sans interruptions qu'est l'essuie-glaces.

Dossier de conception p12

Le fonctionnement de l'essuie-glace est décrit avec cinq états, contrôlé par une seule entrée : le bouton poussoir BP_EssuieGlace.

Au démarrage, le système est dans l'état initial « Essuie-glace OFF », où le moteur est à l'arrêt. Lorsqu'un appui est détecté sur le bouton, le système quitte cet état et passe brièvement par un état intermédiaire avant d'activer l'essuie-glace en mode « vitesse lente ». Dans cette configuration, le balayage s'effectue à une cadence réduite.

Si l'utilisateur appuie de nouveau sur le bouton, le système évolue vers un état « vitesse rapide », dans lequel la fréquence de balayage est augmentée afin d'améliorer la visibilité en cas de fortes précipitations. Enfin, un dernier appui permet de revenir à l'état initial « OFF », stoppant ainsi complètement le fonctionnement de l'essuie-glace.

4.1.2-Description modèle dynamique avec interruption.

Dans nos dossiers conception, nous pouvons observer notre modèle dynamique avec interruptions (**p.12 Dossier_de_Conception**).

Celui-ci correspond au comportement dynamique du système de commandes des clignotants d'une voiture, structuré autour d'une interruption.

Le modèle fonctionne selon un cycle répétitif déclenché par une interruption toutes les 0,25 ms (soit une fréquence de 4 kHz).

Le modèle suit une structure if / else if / else, ce qui garantit qu'une action est prioritaire à la fois, cela évite, d'avoir les deux clignotants activés.

- Sécurité : Chaque cercle (état) force l'extinction de l'autre côté, ce qui assure qu'il n'y a pas de conflit électrique ou visuel.
- Réactivité : Avec un rafraîchissement toutes les 0,25 ms, le système est extrêmement réactif. Le délai entre l'action du conducteur sur le levier et la réponse du système est inexistant.

Lors de l'implantation de ce système au semestre 3, nous avons réalisé ce code sans fonction, avec des HAL_Delay() dans la boucle while du main.c, cela était brouillon.

Cependant, nous avons noté comme amélioration, la mise en place d'une fonction possédant notre code qui serait appelé lors d'une interruption. Cela empêchera les blocages de programme comme nous l'avons vu lors du semestre dernier.

Pour justifier la conception de notre timer, nous avons repris les Delay() que nous avons mis dans le main.c, c'est-à-dire des Delay de 250ms, car avec cette fréquence notre système fonctionnait parfaitement, nous donnant une fréquence de 4Hz, avec notre fréquence d'horloge réglé à 4MHz.

Cela est l'une de nos améliorations parallèles à nos nouveaux systèmes.

5.0-Mise en place de la communication I2C

Cette section présente notre démarche pour concevoir implanter et vérifier la communication I2C entre la STM32 et les deux composants : le thermomètre LM75 et l'écran LCD ST7032.

5.1-Justification des choix de conception

Dans ce projet, nous avons veillé à déterminer, calculer et justifier l'ensemble des paramètres de configuration afin de garantir un fonctionnement fiable et en adéquation avec les contraintes techniques.

Cette démarche m'a également permis montrer une compréhension claire et maîtrisée des choix effectués tout au long de la conception.

Preuve Dossier de Conception

Adresses des composants I2C

Pour concevoir un système comportant des composants I2C, il est obligatoire de connaître leurs adresses, sans elles nous ne pouvons pas communiquer avec les composants.

Celle-ci nous permet de transmettre correctement les trames. Voici comment je les ai déterminées à partir des documentations techniques :

Composant	Adresse 7bits	Adresse 8b écriture	Adresse 8b lecture	Broche ADDR
LM75	0x48	0x90	0x91	A2=A1=A0=GND
ST7032	0x3E	0x7C	--	SA0=GND

Justification de l'adresse du LM75 : l'adresse de base est 0x48. En décalant d'un bit à gauche et en ajoutant le bit R/W (0 pour écriture, 1 pour lecture) : $0x48 \ll 1 = 0x90$ (écriture) et $0x90 | 0x01 = 0x91$ (lecture).

Justification de l'adresse du ST7032 : selon la datasheet, l'adresse par défaut (SA0=0) est 0x3E, soit en 8 bits écriture : $0x3E \ll 1 = 0x7C$.

Clignotants

Par exemple, pour les clignotants, afin de déterminer la valeur des paramètres de l'interruption avec l'ARR et le PSC, nous avons réalisé leur conception avec la formule :

$$f_T = \frac{f_{clock}}{(PSC+1)*(ARR+1)}$$
 avec ici, notre f_{clock} valant 4MHz et une période de 250ms, on impose PSC puis, on peut en déduire le ARR.

Preuve Dossier de Conception

5.2-Utilisation des documentations techniques.

La réalisation de la communication I2C a débuté par une lecture approfondie des documentations techniques de chaque composant (LM75 et ST7032), car sans ces informations, aucune communication n'est possible.

Nous avons recherché leurs adresses dans la documentation, puis pour l'écran nous avons procédé à son initialisation permettant à celui-ci de s'allumer

5.2.1-Adresses I2C issues des datasheets

Le LM75 dispose de trois broches d'adressage (A2, A1, A0). Dans notre montage, ces trois broches sont connectées à GND, ce que la datasheet nous indique directement : l'adresse 7 bits résultante est 0x48. Sur 8 bits, cela donne 0x90 en écriture et 0x91 en lecture. Dans notre cas la bibliothèque HAL gère automatiquement ce décalage.

Pour l'écran LCD ST7032, la datasheet indique une adresse 7 bits fixe de 0x3E, soit 0x7C sur 8 bits en écriture. Sans cette lecture, il nous aurait été impossible d'écrire la bonne valeur dans les appels HAL_I2C_Master_Transmit().

5.2.2-Sémantique des registres

La datasheet du LM75 décrit un registre de température à l'adresse 0x00, contenant 2 octets : le MSB pour la partie entière de la température et le LSB pour le bit de demi-degré. C'est cette lecture qui nous a permis d'écrire correctement la fonction `lm75_lire_temperature()` : envoyer d'abord 0x00 pour pointer vers ce registre, puis lire les 2 octets et les combiner en un `int16_t` avant de décaler de 7 bits pour extraire la valeur en °C.

Pour le ST7032, la datasheet définit un Control Byte qui précède chaque octet envoyé : 0x00 indique que l'octet suivant est une commande de configuration (RS=0), et 0x40 indique que c'est une donnée à afficher (RS=1). Sans cette lecture, l'écran n'aurait jamais répondu correctement.

5.3-Norme de configuration.

Notre système devant assurer une communication fiable entre le microcontrôleur STM32 et les périphériques via le bus I2C, tout en respectant les standards de configuration pour garantir la compatibilité et la maintenabilité de notre projet.

Par exemple, lors de la configuration du STM32, j'ai utilisé CubeMX* pour paramétrer le bus I2C en mode standard et vérifier automatiquement les timings. J'ai également validé la communication avec un oscilloscope pour m'assurer du respect des signaux SDA/SCL.

****=Nous avons été contraints de nous servir de CubeMX, car les ordinateurs avaient subi la mise à jour STM 1.20 et donc le .ioc s'ouvrait et se configurait depuis CubeMX et non CubeIDE.***

Ainsi, le respect des normes de configuration I2C garantit une communication fiable, conforme aux standards industriels, et facilite l'intégration de nouveaux périphériques dans notre projet.

6.0-Modèle fonctionnel.

L'implantation de nos systèmes demandent énormément « d'organisations ». Nos composants I2C, possèdent une initialisation avec un nombre important de fonctions que l'on peut utiliser.

Avec toutes ces fonctions, nous ne pouvons pas nous permettre de réaliser comme au semestre dernier, c'est-à-dire du code dans le while sans nom de fonctions. Nous devons assurer la conception d'un programme propre lisible et bien structuré, permettant la maintenance et durabilité de notre projet dans le temps.

Pour cela, comme nous venons de le voir nous avons déjà réorganiser nos anciennes fonctions. Nous avons créé des fonctions pour chaque système du premier semestre à savoir : clignotants ; appels de phares ; essuie-glace.

Nous avons réalisé la conception de petit fichier code pour chacun des systèmes possédant la fonction du système, cela permet de retrouver les fonctions dans un seul fichier code, si l'étude de ces fonctions est demandée.

Preuve dans les « Fichier de Code » et dans le Dossier de Conception.

Pour la communication I2C, la création de fichiers i2c.c et i2c.h se réalise toutes seul à la suite de l'activation du bus I2C1 sur le STM32CubeMX.

Nous allons voir la méthode de découpage fonctionnel que nous avons utilisé pour organiser notre code, ainsi nous observons aussi les conventions de codage que nous avons respecté dans notre programme informatique.

Nous allons donc épargner les définitions des fonctions i2c et leurs codes, cela se trouve à la pages 17 du dossier de conception.

6.1-Méthode de découpage fonctionnelle.

Nos systèmes devenant de plus en plus complexe, nous devons concevoir un programme simple, afin de garantir la maintenabilité du projet par un tiers, nous privilégions une architecture claire et structurée. En évitant l'accumulation de fonctions complexes et en segmentant le code, nous permettons à un nouvel opérateur de s'approprier rapidement la logique du système. Permettant une maintenance au projet par n'importe quel opérateur.

Dans le cadre de notre projet, nous avons appliqué cela au composants I2C, c'est-à-dire, au capteur de température LM75, à l'écran LCD ST7032.

Je vais expliquer brièvement ces fonctions et pourquoi nous les avons « découpés ».

6.1.1-Gestion du capteur LM75

Nous avons créé des fonctions dédiées au capteur, dans le but de rendre le projet de structurer notre projet.

lm75_init()	lm_75_lire_temperature
Initialise le capteur une seule fois au démarrage Configure la communication I2C	Lit la température Convertir les données brutes en valeurs exploitable

Nous avons séparé l'initialisation et la lecture, cela permet d'éviter les répétitions et d'avoir un code plus propre. L'initialisation se réalisant au début du code, sa séparation avec la lecture de température était obligatoire.

6.1.2-Gestion de l'écran LCD ST7032.

Dans la conception des fonctions de notre écran, nous les avons découpés pour les mêmes raisons que précédemment.

ST7032_Init()	Initialise complètement l'écran
ST7032_Write_Cmd()	Envoie une commande au LCD
ST7032_WriteData()	Envoie un caractère à afficher
ST7032_Print	Affiche une chaîne de caractère
ST7032_SetCursor()	Positionne le curseur

Nous découpons nos fonctions de manière que chacune d'entre elle correspond à une action précise du LCD.

Nous respectons le principe d'une fonction =une tâche.

6.1.3 Fonction globale d'affichage.

EcranI2C()	Effacement de l'écran Affichage de la température Affichage des vitesses.
------------	---

Nous avons ici une fonction globale d'affichage ici, que nous pouvons appeler une fonction de haut niveau, qui utilise elle-même les fonctions de bas niveau (LCD+capteurs).

Notre programme vérifie la méthode de découpage fonctionnel. Chaque fonctionnalité (lecture du capteur, gestion de l'affichage, communication I2C) a été isolée dans des fonctions spécifiques. Cette organisation permet d'obtenir un code structuré, et facilement maintenable. Les fonctions de bas niveau assurent les opérations élémentaires (écriture I2C, affichage d'un caractère), tandis que les fonctions de haut niveau réalisent le comportement global du système.

Preuve dans le Dossier de Conception et dans les fichiers Code

6.2-Respectant les conventions de codage.

Dans la conception de notre programme informatique, nous devons produire un programme lisible, compréhensible par d'autres utilisateurs et maintenable dans le temps.

C'est pour cela que nous devons respecter certaines conventions de codage comme ; le nommage des fonctions et variables ainsi que les commentaires et la structure du code. Nous pouvons notamment associer cela avec le découpage fonctionnel que nous venons de voir.

Preuve dans le Dossier de Conception et dans les fichiers Code

6.2.1 Convention de nommage.

La programmation a requis l'utilisation de nom explicite pour chacun des fonctions des composants I2C, dans le but que n'importe quel utilisateur comprenne son rôle.

Im75_init()	Im_75_lire_temperature	ST7032_WriteCmd()	ST7032_SetCursor
Initialise le capteur LM75	Lit la température	Envoie une commande	Positionne le curseur

Nous pouvons voir que le nom des fonctions décrit clairement leur rôle, à leur nom nous avons une compréhension rapide du code et ce qu'elles permettent de réaliser.

Preuve dans le Dossier de Conception et dans les fichiers Code

6.2.1 Commentaire de code.

Pour permettre une compréhension du code par n'importe quel opérateur effectuant de la maintenance sur celui-ci, nous avons commenté toutes nos fonctions et ce qu'ils se passaient à l'intérieur.

Preuve dans le Dossier de Conception et dans les fichiers Code

Nous avons décrit le rôle de ses fonctions ainsi que les paramètres qu'elles recevaient, pourquoi ces paramètres et son fonctionnement.

Cela permet la compréhension de son rôle et ce qu'elle réalise, donc n'importe quel utilisateur peut comprendre nos fonctions avec l'aide des documentations.

**(On retrouve le code dans SAé_Code et le documentations dans
>SAé_Livrable>ANNEXE>DATASHEET)**

6.2.2 Structure cohérente du code.

Nous pouvons observer dans le programme informatique que nous avons regroupé les fonctions courtes ainsi que les fonctions principales.

Par exemple, nous retrouvons les fonctions courtes comme ST7032_SetCursor() qui regroupent une seule tâche en une fonction, cela nous permet d'obtenir un code aéré avec des blocs bien délimités.

Et inversement nous avons la fonction EcranI2C() qui elle prends dans sa fonction les fonctions courte et réalise une tâche précise. Ici elle réalise l'affichage de la température et de l'accélérations.

Nous pouvons donc dire que le programme respecte les conventions de codage à travers l'utilisation de noms de fonctions explicites, une structure cohérente et des commentaires de documentation détaillés. Cette approche garantit une bonne lisibilité du code, facilite sa maintenance et permet à un autre utilisateur de comprendre rapidement son fonctionnement.

7.0-Filtre numérique

Nous allons passer à la démarche de la conception de notre filtre, et la caractérisation du filtrage obtenu.

7.1-Conception du filtre numérique passe bas du premier ordre

Pour réaliser la conception du filtre numérique, nous partons du filtre passe bas analogique que nous avons implanter lors du semestres3.

Ici nous avons la fonction de transfert pour un filtre passe-bas du premier ordre ; $H(p) = \frac{w_c}{(p+w_c)}$, avec notre fréquence de coupure $F_c=600\text{Hz}$, car au delà de cette fréquence les signaux ne nous intéressent pas.

Pour le transformer en numérique nous avons utilisé la transformé bilinéaire qui remplace p par

$$p = \left(\frac{2}{T_e} * \frac{1-z^{-1}}{1+z^{-1}} \right).$$

Après cette substitution et mise au même dénominateur, nous obtenons $H(z)$ sous forme normalisé, nous permettant d'identifier les trois coefficients du filtre récurrent :

$$b_0 = b_1 = \frac{w_c}{A + w_c} = 0.1586, \text{ et } a_1 = \frac{w_c - A}{A + w_c} = -0.6828$$

Avec $F_e = 10\text{KHz}$, $A = \frac{2}{T_e} = 20\,000$ et $w_c = 2\pi * 600 = 3769\text{rad/s}$

Nous avons l'équation récurrente finale est :

$$y[n] = 0.1586x[n] + 0.1586x[n - 1] + 0.6828y[n - 1]$$

C'est cette équation que nous avons implémenter dans le code STM32.

7.2-Caractérisation du filtre obtenue

Dans le but de réaliser le diagramme de Bode, nous calculons les coefficient α et β (**Voir >SAé_Livrable>Calcul_Filtre_Numérique>CalculCoefficient**), on a :

$$\alpha_0 = 23\,769, \alpha_1 = 16\,231, \beta_0 = \beta_1 = 3\,769$$

Nous pouvons donc observer le Bode à la **page 24 du dossier de conception**.

(Le PDF est disponible dans la section

>SAé_Livrable>Calcul_Filtre_Numérique>Matlab_Diagramme_Bode)

Le diagramme obtenu nous montre bien la pente de -20dB/décade . Nous pouvons aussi notifier que la fréquence de coupure apparait est à 300Hz au lieu de 600Hz . Nous ne comprenons malheureusement pas encore pourquoi, mais nous pouvons penser à un problème de normalisation dans Matlab.

Hormis cette confusion sur Matlab, nous pouvons notifier le bon fonctionnement de notre filtre passe-bas sur la STM32 comme nous pouvons le voir à la vidéo (**cf. VidéoFiltre**).

Sur la vidéo, nous pouvons observer le fonctionnement du filtre a $0.1f_c$ avec un fonctionnement parfait, nous retrouvons bien le sinus allant de 0V à 2V en sortie, puis nous avons mis la fréquence de f_c , nous pouvons observer un sinus atténué, et enfin, lorsque nous mettons une fréquence de $10f_c$, nous pouvons observer une amplitude totalement atténuée avec une valeur de 1.

(Les capture d'écran sont disponibles dans la section

>SAé_Livrable>Calcul_Filtre_Numérique>_PicoScope_Filtre_Passe_Bas)

Nous constatons donc que hormis le diagramme de Bode, notre filtre est bien fonctionnel physiquement sur la carte STM32, avec une atténuation du sinus qui débute lorsque nous atteignons f_c .

7.3-Débogage STM32.

L'implantation de ce filtre sur STM32, nous a été critique, car celle-ci est le système qui nous a demandé le plus de temps et d'énergie, nous avons eu beaucoup de difficulté à faire fonctionner ce filtre sur STM32.

Pour remédier à cela, nous avons réalisé une manière de débogage particulière ; la méthode de pas à pas.

Nous avons débogué notre projet via l'icône , celle-ci nous permet de voir la valeur des variables souhaités en permanence et ce à chaque tour de boucle/interruptions.

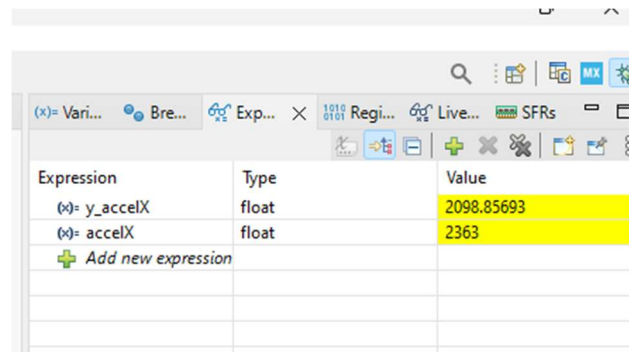
Nous allons donc nous en servir pour voir nos valeurs du filtre et voir si avec ces valeurs la nôtre sortie serait cohérente, si elles le sont, nous pourrions donc chercher une autre erreur possible qui n'est pas liée au code, soit le câblage, le câble, la carte.....

Nous avons donc appliqué la méthode avec 2 de nos variables, `y_accelX` et `acceX`.

Nos 2 variables sont acquises avec la fonction `HAL_ADC_GetValue(&hadc);`, donc ce sont des variables allant de 0 à 4095.

Nous déboguons avec cette méthode, et nous trouvons ;

Nous pouvons donc observer que nos 2 valeurs sont cohérentes car elles sont comprises entre 0 et 4095 et en plus celle-ci sont au milieu de ces valeurs, ce qui correspond avec notre sinus de 2V que nous avons envoyé en entré et notre STM32 a une plage allant jusqu'à 3.3Volts.



Expression	Type	Value
(x)= y_accelX	float	2098.85693
(x)= acceX	float	2363
+ Add new expression		

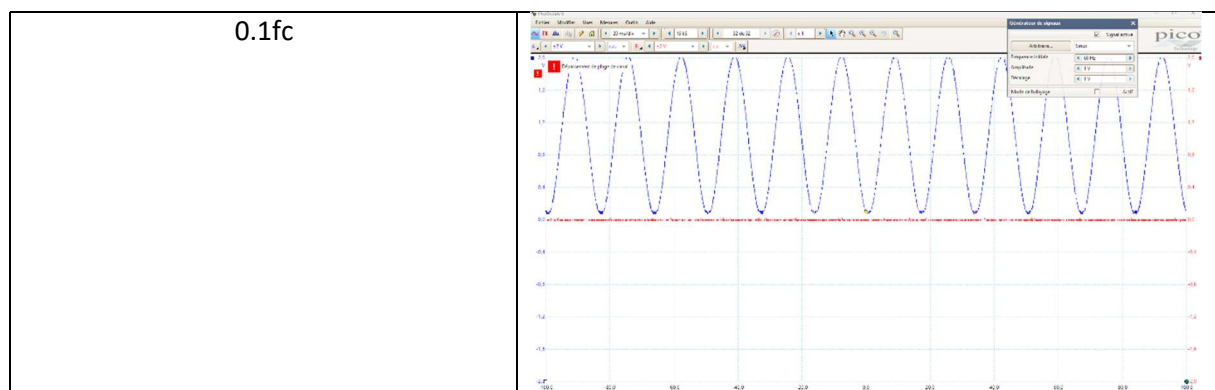
Cette méthode de débogue, nous permet donc d'analyser le problème, celui-ci ne se trouve pas dans notre code, car notre fonction (ci-dessous) possède donc une valeur cohérente.

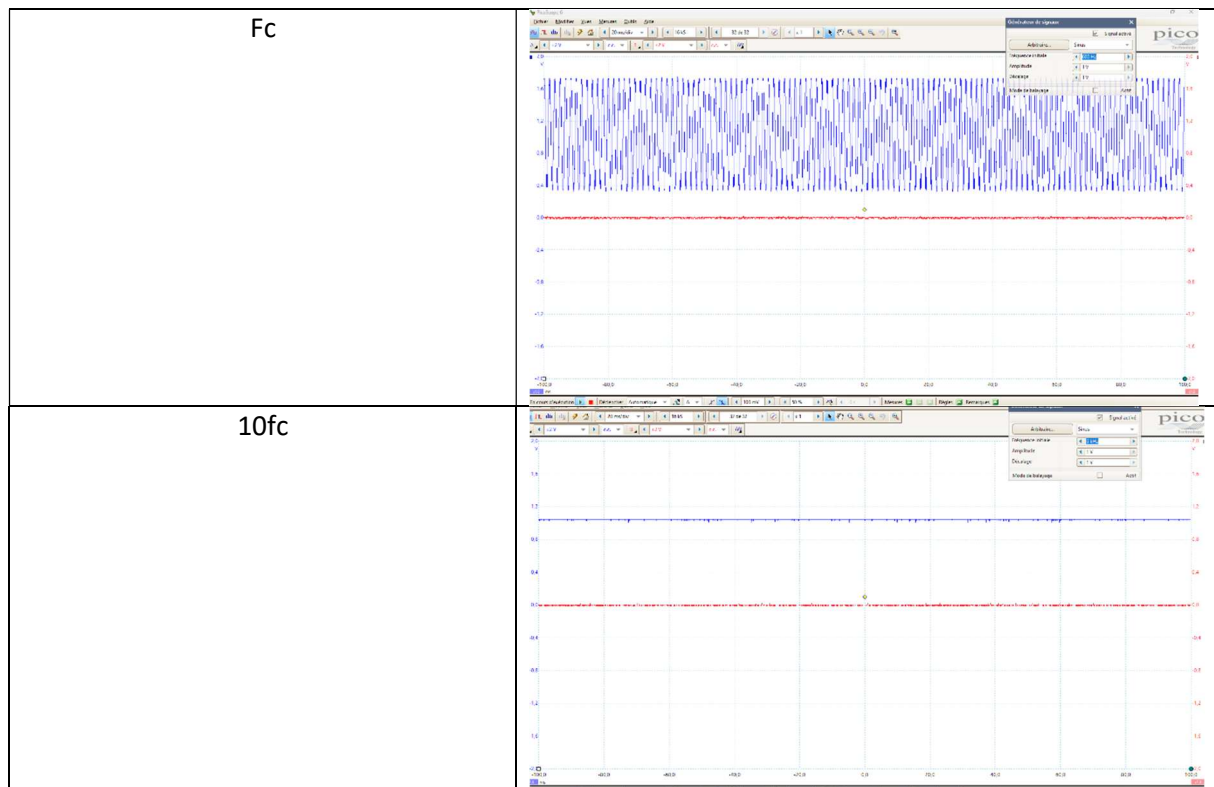
```
HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, (uint16_t)y_accelX);
```

Nous pouvons donc vérifier le branchement du circuit au PicoScope ou tester la fonctionnalité, du câbles et carte STM32.

Après avoir analyser le câblage, nous avons inverser la sonde SCL et SDA, une fois celle-ci corriger, nous pouvions observer le fonctionnement de notre filtre sur le PicoScope avec les différentes fréquences, on prend 0.1fc, fc et 10fc (fc=600Hz).

Nous avons donc sur le PicoScopeX ;





Notre filtre est donc fonctionnel, nous avons réussi à le faire fonctionner grâce à la méthode de débogage qui nous a montré que nos valeurs de `accel_X` étaient justes.

Nous avons saisi l'efficacité de cette méthode de débogage, qui permet d'observer en temps réel la valeur de n'importe quelle variable et ainsi de localiser plus facilement l'origine d'un problème dans le code.

7-4. Maîtrise des temps d'exécution.

En système embarqué la gestion du temps est cruciale. Comme évoqué plus tôt, au semestre 3, notre façon de coder les clignotants était brouillon et ne permettait pas un fonctionnement parfait et précis.

C'est avec ces erreurs que nous avons réalisé l'importance de la maîtrise des temps d'exécution.

Nous avons dû optimiser le temps de calcul et de mettre en place une mesure des temps d'exécution. Voici l'état de notre démarche.

7.4.1-Optimisation par le choix d'un filtre IIR du 1er ordre

Le choix d'un filtre IIR (à réponse impulsionnelle infinie) du 1er ordre est déjà en lui-même un choix d'optimisation. Comparé à un filtre FIR (à réponse impulsionnelle finie) qui nécessiterait N multiplications par échantillon pour obtenir une pente de -20 dB/décade, notre filtre IIR n'en requiert que 3 :

- $b_0 \times x[n]$ – multiplication de l'entrée courante

- $b1 \times x[n-1]$ – multiplication de l'entrée précédente
- $a1 \times y[n-1]$ – multiplication de la sortie précédente (terme récursif)

Cela représente 3 multiplications flottantes et 2 additions par axe, soit 6 multiplications et 4 additions pour les deux axes X et Y. Sur STM32, une multiplication flottante simple précision prend 1 cycle. L'exécution complète de la fonction ADLXFiltre() est de l'ordre de quelques dizaines de microsecondes, bien en dessous de la période d'échantillonnage $T_e = 100 \mu s$ ($f_e = 10 \text{ kHz}$).

7.4.2-Point d'amélioration identifié : mesure instrumentée du temps

Dans le cadre de l'implémentation d'un filtre numérique passe-bas du premier ordre, nous avons déterminé et justifié les paramètres temporels nécessaires au traitement du signal.

En fixant une fréquence de coupure de 600 Hz et une fréquence d'échantillonnage de 10 000 Hz, nous avons établi une période d'échantillonnage $T_e = 100 \mu s$, tout en vérifiant le respect du critère de Shannon ($f_e \approx 16,7 \times f_c$), garantissant ainsi l'absence de repliement spectral.

Le calcul des coefficients du filtre par la méthode de la transformée bilinéaire m'a permis de dériver l'équation aux différences finale, dont chaque itération doit être exécutée à cadence fixe.

Cette contrainte temporelle impose la configuration d'un timer matériel précis, afin que le filtre s'applique de manière synchrone et déterministe sur le signal numérisé.

8-0. Observations des tests.

Après avoir réalisé la conception, implantation de nos systèmes, nous devons le tester.

Nous devons vérifier s'ils réalisent les exécutions souhaitées, dans le but de valider ou non nos systèmes et donc notre projet.

8-1. Mise en place de test unitaire.

Nous avons mis en place, dans le rapport de test, une série de tests unitaires destinés à vérifier le bon fonctionnement de chaque composant avant leur intégration au sein du système global.

Ces tests ont été réalisés sur différents éléments matériels, notamment les LEDs, le bouton poussoir, les accéléromètres, le commutateur ainsi que l'écran LCD. Par ailleurs, des tests unitaires ont également été développés pour chacun des sous-systèmes définis dans le cahier des charges. Chaque sous-système dispose ainsi de son propre fichier de code STM32, contenant uniquement les fonctions qui lui sont spécifiques.

Ces tests unitaires présentent plusieurs avantages. Ils permettent notamment à un opérateur découvrant le projet de valider rapidement le bon fonctionnement des

composants, en s'appuyant sur le dossier de fabrication, avant leur assemblage. Ils contribuent également à garantir la fiabilité du système global, en assurant que chaque composant fonctionne correctement et qu'aucun élément n'est défectueux.

Preuves dans Rapport de Test/Procédure de Test et dans les fichiers code.

8-1. Observations des trames en vérifiant le respect du protocole.

Afin de valider la conformité des communications entre le microcontrôleur STM32 et les périphériques I2C du système, nous avons procédé à une analyse systématique des trames échangées sur le bus.

Pour chaque composant — le capteur de température LM75 (adresse 0x48) et l'écran LCD ST7032 (adresse 0x3E) — nous avons défini théoriquement les trames attendues, en détaillant chaque champ : condition START, adresse esclave avec bit R/W, octets de contrôle, données et acquittements. Ces trames de référence ont ensuite été comparées aux trames réellement capturées à l'oscilloscope et à l'analyseur logique sur les lignes SDA et SCL.

Cette démarche nous a notamment permis d'identifier et de corriger un bug dans la fonction de lecture du LM75, où la valeur de température retournée correspondait au double de la valeur réelle faute d'un décalage binaire manquant. La comparaison trame par trame constitue ainsi une méthode rigoureuse de débogage bas niveau, complémentaire aux tests fonctionnels de plus haut niveau.

Cette phase de vérification nous a permis de détecter une erreur dans notre implémentation initiale : un décalage de 8 bits au lieu de 7, ce qui doublait la valeur obtenue. Après correction, la valeur affichée a été validée en la comparant à la température réelle mesurée dans la salle.

9-Plan de validation du système.

9.1-Schéma de câblage et justification

Pour notre projet, nous avons réalisé le schéma électrique suivant, possédant tous les composants du système.

Le schéma électrique se trouve sur le Drive dans ->SAé_Livrable->Annexe->Schéma électrique

Sur notre schéma électrique nous retrouvons bien tous les systèmes et composants qui sont disponibles sur notre projet.

Comme système nous y retrouvons ; les appels de phares, les clignotants, les essuie-glaces, l'affichage de l'accélération avec l'acquisition des données depuis l'ADLX-335 jusqu'à son affichage sur les composants I2C écran LCD ST7032 via la communication I2C avec aussi la communication entre le capteur de température LM75 et l'écran nous permettant l'affichage de la température en temps réel.

9.2-Matériel et environnement de test

Pour ce projet, nous avons utilisé une longue liste de composants disponible dans le **dossier de fabrication et conception**, nous y retrouvons sa référence, désignation avec ses caractéristiques, le nombre de composants nécessaire pour le fonctionnement de notre système.

Tous les composants du tableau sont à tester et ONT été tester. Nous retrouvons le résultat dans **le rapport de test**.

9.2.1-Justification des étapes.

Chacun des tests à été réalisé de la même façon dans le but d'être le plus précis possible.

Nous avons réalisé des tests pour chaque composant avant de les assembler et que cela nous donne notre système final. *Nous ne souhaitons pas avoir un composant défectueux qui nous mènent en erreur sur le fonctionnement global de notre projet.*

Dans **le Rapport de Test**, nous avons réalisé un test pour chaque composant ; LEDs, bouton poussoir, commutateur, accéléromètres. Ces tests expliquaient comment nous les avons tester afin de pouvoir réalise de nouveau ces test, mais aussi les condition de validation afin d'assurer le bon fonctionnement de chaque composants.

Cela nous a permis d'assurer le fonctionnement de chaque composant et donc assurer le fonctionnement global de notre projet.

Pour illustrer concrètement la démarche de test mise en place, prenons l'exemple de la validation de la communication I2C entre la carte STM32 et ses deux périphériques : le capteur de température LM75 et l'écran LCD ST7032.

Dans un premier temps, nous avons réalisé le câblage complet du système en nous appuyant sur le schéma électrique défini en amont, en reliant notamment les lignes SDA et SCL de chaque composant aux broches PB9 et PB8 de la STM32, tout en veillant à la présence des résistances de pull-up indispensables au bon fonctionnement du bus I2C. La carte a ensuite été connectée au PC via USB afin de permettre le téléversement du programme depuis STM32CubeIDE.

Une fois le code compilé sans erreur et chargé sur la carte, nous avons observé le comportement du système : l'écran LCD devait s'allumer et afficher en temps réel la température ambiante relevée par le LM75. Pour valider non seulement le résultat visible, mais aussi la conformité des échanges au niveau protocolaire, nous avons branché un analyseur logique sur les lignes SDA et SCL afin de capturer les trames réelles et de les comparer aux trames théoriques attendues.

Cette vérification nous a permis de nous assurer que les adresses I2C étaient correctement reconnues (0x48 pour le LM75, 0x3E pour le ST7032), que les acquittements étaient bien présents à chaque octet transmis, et que les données reçues correspondaient aux valeurs théoriques définies dans notre document de référence.

9.3-Justification des plages de validation.

Pour chaque composant du système, nous avons défini des plages de validation précises basées sur les modèles théoriques et les datasheets.

Pour l'accéléromètre ADXL335, la tension de sortie théorique suit la relation $V_{out} = 0,6 + (0,3 \times a)$, ce qui impose une plage comprise entre 0 V et 1,65 V pour une accélération de 0 à 3,5 g, justifiée par la sensibilité de $300 \text{ mV} \cdot \text{g}^{-1}$ et l'offset de 0,6 V du composant.

Pour le capteur de température LM75, une table de référence des trames I2C attendues a été établie : à 25°C par exemple, `data[0]` doit valoir 0x19 et `data[1]` doit valoir 0x00, ce qui après traitement logiciel donne bien la valeur correcte.

Pour le filtre passe-bas numérique, trois points de mesure ont été définis : à 0,1·fc le signal de sortie doit être quasi identique à l'entrée, à fc l'atténuation doit être de -3 dB (facteur $\sqrt{2}$), et à 10·fc l'atténuation doit atteindre -20 dB, conformément à la pente théorique d'un filtre du premier ordre.

Ces critères chiffrés ont permis de valider le système de manière objective et traçable.

9.4-Respect des règles normatives

Dans le cadre de la conception et de la validation de notre système embarqué, nous avons veillé à respecter les règles normatives en vigueur relatives aux communications et aux interfaces utilisées.

Le protocole I2C a été mis en œuvre conformément à sa spécification standard, notamment en ce qui concerne la présence obligatoire de résistances de pull-up sur les lignes SDA et SCL, l'attribution d'adresses 7 bits uniques à chaque périphérique (0x48 pour le LM75, 0x3E pour le ST7032), et le respect de la séquence START / ADRESSE / ACK / DATA / STOP définie par le protocole. Les niveaux de tension du système ont été maintenus à 3,3 V conformément aux spécifications électriques des composants utilisés, notamment la carte STM32 Nucleo L073RZ et le capteur ADXL335.

Par ailleurs, le développement logiciel a été réalisé en langage C/C++ sous STM32CubeIDE en s'appuyant sur la couche d'abstraction matérielle HAL fournie par STMicroelectronics, garantissant une conformité avec les bonnes pratiques de développement embarqué recommandées par le fabricant.

9.5-Analyse des résultats

L'ensemble des tests réalisés a permis de valider le fonctionnement global du système embarqué, bien que plusieurs sources d'erreurs aient été identifiées au cours de la campagne de validation.

Concernant le capteur de température LM75, un bug a été détecté dans le traitement logiciel : l'absence de la division par 2 lors du décodage des octets MSB et LSB conduisait à retourner une valeur deux fois supérieure à la température réelle, ce qui illustre l'importance de la vérification des trames I2C à l'analyseur logique.

Pour le filtre passe-bas numérique, des écarts entre les valeurs théoriques et mesurées peuvent être imputés aux tolérances de conversion de l'ADC/DAC, aux délais d'exécution du

code sur la STM32, ou encore à la qualité du signal généré par le PicoScope. S'agissant de l'accéléromètre ADXL335, la sensibilité du capteur aux vibrations parasites de l'environnement de test a pu fausser certaines mesures malgré la présence du filtre passe-bas.

En termes de pistes d'amélioration, il serait pertinent d'implémenter un filtre d'ordre supérieur pour une atténuation plus sélective du bruit, de mettre en place une routine de calibration automatique de l'accéléromètre au démarrage, et de renforcer la gestion des erreurs sur le bus I2C afin de détecter en temps réel toute absence de réponse d'un périphérique esclave.

10-Analyse réflexive.

Ce projet m'a énormément apporté, tant sur le plan technique qu'humain, mais avec le recul, je peux identifier clairement ce que j'aurais fait différemment.

La première erreur que je ne reproduirai plus est de négliger la phase de test unitaire : nous avons parfois téléversé directement le code complet sur la STM32 sans tester chaque composant indépendamment, ce qui a rendu le débogage bien plus fastidieux lorsqu'un problème survenait.

J'aurais également pris davantage de temps en amont pour maîtriser le protocole I2C en profondeur avant de commencer le développement, car c'est précisément ce manque de rigueur initial qui a conduit au bug de lecture du LM75, un problème qui m'a coûté un temps précieux et qui aurait pu être évité avec une lecture plus attentive de la datasheet.

Sur la partie traitement du signal, je pense que j'aurais pu aller plus loin en implémentant un filtre d'ordre supérieur dès le départ, plutôt que de partir sur un premier ordre et de se rendre compte de ses limites en phase de test.

Plus globalement, ce projet m'a appris que la documentation et la rigueur méthodologique ne sont pas une perte de temps, mais bien le socle sur lequel repose la réussite d'un projet embarqué : chaque heure passée à bien spécifier, bien câbler et bien tester en fait gagner trois en débogage. C'est une leçon que j'emmènerai avec moi dans tous mes projets futurs.

Au-delà des compétences acquises, ce projet a été pour moi bien plus qu'un simple exercice académique : concevoir un ordinateur de bord pour voiture de course, c'était exactement le genre de projet auquel je portais depuis longtemps, porté par une passion pour l'automobile et les systèmes embarqués.